Projeto Final — DAD (Desenvolvimento de Aplicações Desktop)

Contexto

Este é o projeto final da disciplina. Cada grupo vai receber (por entrevista com o professor) a descrição de um sistema pedido por um "cliente fictício" e terá que, a partir disso, levantar os requisitos, projetar e construir uma aplicação desktop completa.

O uso de IA generativa para ajudar a construir o projeto é permitido e esperado — mas o trabalho precisa ter complexidade suficiente para que copiar e colar respostas de uma IA sem entender o resultado não seja uma opção viável. Por isso os requisitos abaixo são mínimos obrigatórios, não sugestões.

Requisitos técnicos obrigatórios

1. **Banco de dados relacional com no mínimo 6 tabelas relacionadas entre si** — relacionamentos reais (chaves estrangeiras), não tabelas soltas sem nenhuma ligação entre elas.
2. **CRUD completo** (Create, Read, Update, Delete) para todas as entidades do sistema, não só para a tabela principal.
3. **Consumo de dados de API em pelo menos 4 situações distintas** dentro do fluxo do app. Pode ser API pública já existente, API própria feita pelo próprio grupo, ou uma combinação das duas — mas as 4 situações precisam ser genuinamente diferentes entre si (fontes/finalidades diferentes), e cada uma precisa ter um propósito real dentro do sistema, não existir só para cumprir o requisito.
4. **Segurança.** O grupo precisa pensar em segurança como parte do projeto, não como item esquecido no fim — por exemplo: senhas nunca em texto puro, validação de entradas, controle de acesso adequado ao domínio do sistema, cuidado com dados sensíveis. Cada grupo deve justificar as decisões de segurança que tomou para o seu sistema específico.
5. **Código modularizado em classes.** Lógica de negócio organizada em classes (POO), não funções soltas e desorganizadas.
6. **Aplicação desktop em Electron, com TypeScript no lugar de JavaScript.** Toda a lógica da aplicação deve ser escrita em TypeScript (sem arquivos `.js` de lógica da aplicação).

Uso de IA generativa

O uso de IA para ajudar a arquitetar e construir o projeto é incentivado, desde que o grupo entenda e saiba explicar qualquer decisão tomada com ajuda dela. Recomendações:

- Usar o **OpenCode** (ou outro cliente de IA local equivalente) para **arquitetar o projeto antes de começar a codar** — não só pedir código pedaço por pedaço sem plano.
- Considerar a criação de **agentes** especializados e **skills** reutilizáveis para as tarefas recorrentes do projeto.
- Avaliar, já na fase de arquitetura, se o projeto precisa de:
  - **MCP**, caso seja necessário conectar a aplicação a algum serviço ou aplicação externa específica;
  - **RAG**, caso seja necessário que a IA consulte documentos específicos do domínio do projeto.

Essas são recomendações de como tirar melhor proveito da IA como ferramenta de desenvolvimento — não dispensam o grupo de entender a arquitetura final do próprio sistema.

Entrega

- A entrega é feita via **GitHub**, em um repositório do grupo.
- O usuário **willcotil** deve ser adicionado como membro/colaborador do repositório desde o início do projeto, não só na entrega final.
- O histórico de commits será avaliado: é necessário commitar **cada etapa** do desenvolvimento ao longo do tempo, não entregar tudo em um único commit no final.
- **Todos os membros do grupo devem ter commits próprios**, identificáveis pelo autor do commit. A participação de cada integrante será avaliada também pelo histórico de commits, e não apenas pelo resultado final entregue.